

# Unit B1: Computational Thinking

**Sub-topic:** B1.1 Approaches to Computational Thinking

## 1. Syllabus Scope & Learning Outcomes

*This section defines the exam boundaries. The AI must prioritize these verbs.*

| Syllabus Point | Command Term     | Student Expectation (The "Do")                                                                   |
|----------------|------------------|--------------------------------------------------------------------------------------------------|
| B1.1.1         | <b>Construct</b> | Create a detailed problem specification including 5 specific components.                         |
| B1.1.2         | <b>Describe</b>  | Define and characterize Abstraction, Decomposition, Pattern Recognition, and Algorithmic Design. |
| B1.1.3         | <b>Explain</b>   | Connect the four concepts above to specific CS scenarios (e.g., Machine Learning, Networks).     |
| B1.1.4         | <b>Trace</b>     | Follow a flowchart step-by-step to track variable values and determine output (Trace Tables).    |

## 2. Key Terminology & Definitions

*Core vocabulary students must define or identify in short-answer questions.*

- **Computational Thinking:** A toolkit of techniques for problem-solving (not just programming).
- **Problem Specification:** A short, clear explanation of an issue.
- **Stakeholder:** Individuals/groups inside or outside an organization affected by a software project.
- **Decomposition:** Breaking down complex problems into smaller, manageable parts (supports modularity).

- **Abstraction:** Extracting essential information while ignoring irrelevant details (hiding complexity).
- **Pattern Recognition:** Identifying similarities in problems to facilitate code reuse and efficiency.
- **Algorithm:** A finite sequence of step-by-step instructions to solve a problem.
- **Flowchart:** A diagrammatic representation of an algorithm using standard symbols.

### 3. Core Content & Exam Actions

*Detailed breakdown for slide generation. Focus on the "Student Exam Action" bullets.*

#### B1.1.1: Construct a Problem Specification

**Theory:**

- Computers require precise instructions; specifications bridge the gap between human needs and code.
- **The 5 Mandatory Components of a Specification:**
  1. **Problem Statement:** Description of the issue, target audience, and current deficits.
  2. **Constraints & Limitations:** Hardware/software limits, budget, legislation (GDPR), time, workforce.
  3. **Objectives & Goals:** What needs to be achieved/solved.
  4. **Input/Output Specifications:**
    - *Inputs:* Direct (scanners), Manual (keyboard/mouse), Automatic (sensors).
    - *Outputs:* Temporary (screen), Permanent (print), Mechanical (actuators).
  5. **Evaluation Criteria:** Measurable metrics to judge success later.

**Student Exam Action (Command: Construct):**

- Students must be able to read a scenario (e.g., "A school library system") and draft a specification containing the 5 components above.
- *Constraint Checking:* They must identify specific constraints (e.g., "Must run on existing tablets," "Budget under \$500").

#### B1.1.2: Fundamental Concepts

**Theory:**

- **Abstraction:** Used in maps (tube maps), paintings, and OOP (hiding implementation details). Reduces complexity, file size, and dev time.
- **Decomposition:** Breaks problems down. **Correction:** It is breaking down the *problem*, not just breaking a program into subroutines (a common student error).
- **Pattern Recognition:** Leads to reusable code (functions/libraries) and faster development.
- **Algorithmic Design:** Creating predictable, sequential steps.

**Student Exam Action (Command: Describe):**

- Students must define these terms accurately.
- Students must distinguish between them in a given scenario (e.g., "Which concept is being applied when a programmer ignores the CPU architecture to write Java? -> Abstraction").

### B1.1.3: Applying Concepts to CS Domains

#### Theory:

- **Software Dev:** Decomposition for modularity; Abstraction for libraries.
- **Machine Learning:** Pattern recognition (classifying data/predicting trends).
- **Networks:** Decomposition (breaking down security models); Pattern recognition (identifying threats/virus signatures).
- **Databases:** Abstraction (viewing data models); Decomposition (normalizing tables).

#### Student Exam Action (Command: Explain):

- Students must link a concept to a *non-programming* context if asked (e.g., "How is abstraction used in Network Security?").

### B1.1.4: Flowcharts & Tracing

#### Theory:

- **Standard Symbols:**
  - Oval: Start/End (Terminator).
  - Parallelogram: Input/Output.
  - Rectangle: Process (Calculation/Assignment).
  - Diamond: Decision (Yes/No).
  - Arrows: Flowlines.

#### Student Exam Action (Command: Trace):

- **Trace Tables:** Students must create a table with columns for variables and output.
- **Execution:** They must step through the logic row-by-row, updating variables only when changed.
- **Logic:** They must handle loops (iteration) and IF statements (selection) correctly within the trace.

## 4. Common Pitfalls & Examiner Tips

*Crucial for "Exam Prep" resources.*

- **Pitfall (Decomposition):** Students often define decomposition as "breaking a program into sub-programs." **Correction:** It is breaking the *problem* into parts *before* programming begins.
- **Pitfall (Flowcharts):** Students often forget to label the "Yes/No" or "True/False" branches on Decision diamonds.
- **Tip (Trace Tables):** When a variable value does not change in a step, leave the cell blank

or repeat the value—consistency matters, but clarity is key.

- **Tip (Constraints):** In "Construct" questions, generic constraints (e.g., "fast computer") score poorly. Specific constraints (e.g., "Must be compatible with iOS 15") score well.